

Name: Gunturu Rahul
Regd.no: 2024106242

AI Lab Week 2

Monkey Banana Problem

CODE:

```
[ ]: from collections import deque

[ ]: print("-----INITIAL STATES-----")
    m_position = input("Enter monkey position (A,B,C): ")
    b_position = input("Enter Box position (A,B,C): ")
    banana_position = input("Enter Banana position (A,B,C): ")
    status = 0
    has_banana = False

[ ]: q = deque([(m_position, b_position, status, has_banana)])

    visited = set([(m_position, b_position, status, has_banana)])

    parent = {(m_position, b_position, status, has_banana) : None}

    ans_state = []

[ ]: while q:
    state = q.popleft()

    if state[3]:
        ans_state = state
        break

    gen_states = []

    #move to box
    if state[2] == 0:
        gen_states.append((state[1], state[1], state[2], state[3]))

    #push box
    if state[0] == state[1] and state[2] == 0:
        gen_states.append((banana_position, banana_position, state[2], state[3]))

    #climb
    if state[0] == state[1] and state[2] == 0:
```

```
gen_states.append((state[0], state[1], 1, state[3]))
```

```
#grab
```

```
if state[2] == 1 and state[0] == banana_position:  
    gen_states.append((state[0], state[1], state[2], True))
```

```
for i in gen_states:  
    if i not in visited:  
        visited.add(i)  
        q.append(i)  
        parent[i] = state
```

```
print("Monkey at, Box at, Monkey on, Has banana")
```

```
if ans_state:  
    path = [ans_state]  
    i = ans_state  
    while parent[i]:  
        path.append(parent[i])  
        i = parent[i]  
    path.reverse()
```

```
for i in path:  
    print(i)
```

```
print("Banana Reached")
```

```
else:  
    print("No solution")
```

I/O:

Input - 1:

```
-----INITIAL STATES-----  
Enter monkey position (A,B,C): A  
Enter Box position (A,B,C): B  
Enter Banana position (A,B,C): C
```

Output - 1:

```
Monkey at, Box at, Monkey on, Has banana  
( 'A', 'B', 0, False)  
( 'B', 'B', 0, False)  
( 'C', 'C', 0, False)  
( 'C', 'C', 1, False)  
( 'C', 'C', 1, True)  
Banana Reached
```

Input - 2:

```
-----INITIAL STATES-----  
Enter monkey position (A,B,C): B  
Enter Box position (A,B,C): B  
Enter Banana position (A,B,C): C
```

Output - 2:

```
Monkey at, Box at, Monkey on, Has banana  
( 'B', 'B', 0, False)  
( 'C', 'C', 0, False)  
( 'C', 'C', 1, False)  
( 'C', 'C', 1, True)  
Banana Reached
```

Input - 3:

```
-----INITIAL STATES-----  
Enter monkey position (A,B,C):  A  
Enter Box position (A,B,C):  C  
Enter Banana position (A,B,C):  C
```

Output - 3:

```
Monkey at, Box at, Monkey on, Has banana  
( 'A', 'C', 0, False)  
( 'C', 'C', 0, False)  
( 'C', 'C', 1, False)  
( 'C', 'C', 1, True)  
Banana Reached
```

Input - 4:

```
-----INITIAL STATES-----  
Enter monkey position (A,B,C):  C  
Enter Box position (A,B,C):  C  
Enter Banana position (A,B,C):  C
```

Output - 4:

```
Monkey at, Box at, Monkey on, Has banana  
( 'C', 'C', 0, False)  
( 'C', 'C', 1, False)  
( 'C', 'C', 1, True)  
Banana Reached
```

Input - 5:

```
-----INITIAL STATES-----  
Enter monkey position (A,B,C):  A  
Enter Box position (A,B,C):   B  
Enter Banana position (A,B,C):  C
```

Output - 5:

```
No solution
```